



AS PER
LATEST
SYLLABUS

MCA-205

Python

PROGRAMMING



By Virendra Goura

```
def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n - 1)  
  
num = int(input("Enter a number: "))  
result = factorial(num)  
print(f"Factorial of {num} is {result}")
```

Code. Learn. Create. Repeat.



EASY TO
UNDERSTAND



PRACTICAL
EXAMPLES



CONCEPTS
MADE SIMPLE



BOOST YOUR
CODING SKILLS



www.virendragoura.com

PREFACE

Dear Reader,

It is a privilege to present this comprehensive collection of RTU MCA Semester Examination Notes, designed to cover the complete syllabus with clarity and academic accuracy. Every concept, definition, explanation, and example has been organized to support thorough understanding and effective exam preparation.

The purpose of these notes is to simplify complex topics and provide a reliable study resource that can assist in both detailed learning and quick revision. Continuous effort has been made to ensure correctness and relevance; however, learning grows when readers engage, question, and explore further.

May these notes serve as a strong academic foundation and contribute meaningfully to your preparation and future growth.

Warm regards,
Virendra Goura
www.virendragoura.com

DISCLAIMER

This e-book has been created with utmost care, sincere effort, and extensive proofreading. However, despite all attempts to avoid mistakes, there may still be some errors, omissions, or inaccuracies that remain unnoticed.

This e-book is issued with the understanding that neither the author nor the publisher shall be held responsible for any loss, damage, or misunderstanding arising from the use of the information contained within.

All content provided is for educational and informational purposes only.

© 2026— All Rights Reserved.

No part of this e-book may be reproduced, copied, scanned, stored in a retrieval system, or transmitted in any form—whether electronic, mechanical, digital, or otherwise—without prior written permission from the author/publisher.

Any unauthorized use, sharing, or distribution of the material is strictly prohibited and may lead to civil or criminal liability under applicable copyright law.

MCA - 205 Python Programming

Unit-1 Introduction and Overview:

Introduction, What is Python, Origin, Comparison, Comments, Variables and Assignment, Identifiers, Basic Style Guidelines, Python Objects, Standard Types, Other Built-in Types, Internal Types, Operators, Built-in Functions, Numbers and Strings. Introduction to Numbers, Integers, Floating Point Real Numbers, Complex Numbers, Sequences: Strings, Sequences, Strings, String-only Operators, Built-in Functions, String Built-in Methods, Special Features of Strings, Memory Management, Python Application Examples.

Unit-2 Lists and Dictionaries:

Built-in Functions, List type built in Methods, Special Features of Lists, Tuples, Tuple Operators and Built-in Functions, Special Features of Tuples, Introduction to Dictionaries, Built-in Functions, Built-in Methods, Dictionary Keys, Conditionals and Loops: if statement, else Statement, elif Statement, while Statement, for Statement, break Statement, continue Statement, pass Statement, else Statement.

Unit-3 Object, Classes and Files:

Classes in Python, Principles of Object Orientation, Creating Classes, Instance Methods, Class variables, Inheritance, Polymorphism, Type Identification, File Objects, File Built-in Function, File Built-in Methods, File Built-in Attributes, Standard Files, Command-line Arguments, File System, File Execution, Persistent Storage Units.

Unit-4 Regular Expression and Exception Handling:

Regular Expression: Introduction/Motivation, Special Symbols and Characters for REs, REs and Python. What Are Exceptions? Exceptions in Python, Detecting and Handling Exceptions. Exceptions as Strings. Raising Exceptions, Assertions, Standard Exceptions.

Unit-5 Database Interaction:

SQL Database Connection using Python, Creating and Searching Tables, Reading and storing config information on database, Programming using database connections, Python Multithreading: Understanding threads, Forking threads, synchronizing the threads, Programming using multithreading.

UNIT-1 Introduction and Overview

Introduction, What is Python, Origin, Comparison, Comments, Variables and Assignment, Identifiers, Basic Style Guidelines, Python Objects, Standard Types, Other Built-in Types, Internal Types, Operators, Built-in Functions, Numbers and Strings. Introduction to Numbers, Integers, Floating Point Real Numbers, Complex Numbers, Sequences: Strings, Sequences, Strings, String-only Operators, Built-in Functions, String Built-in Methods, Special Features of Strings, Memory Management, Python Application Examples.

Introduction

Programming languages are used to communicate instructions to computers. A computer understands only machine language, which is difficult for humans to write and understand. Therefore, high-level programming languages were developed to simplify programming and make software development easier.

Python is one of the most popular high-level programming languages in the world. It is widely known for its simplicity, readability, portability, and powerful capabilities. Python helps programmers write programs using fewer lines of code compared to other programming languages such as C, C++, and Java.

Python is used in many fields such as: Web Development, Artificial Intelligence, Machine Learning, Data Science, Automation, Scientific Computing, Cloud Computing, Cyber Security.

Python is suitable for beginners because of its easy syntax and also powerful enough for advanced software development and research work.

What is Python: Python is a high-level, interpreted, object-oriented, and general-purpose programming language used for software development and problem solving.

Python focuses mainly on: Simplicity, Readability, Productivity.

Python allows developers to create applications quickly because its syntax is similar to English language statements.

Example: `print("Hello Python")`

Features of Python

1. Simple and Easy Language: Python syntax is simple and easy to understand. It uses fewer keywords and less complex syntax.

2. Interpreted Language: Python programs are executed line-by-line using an interpreter.

Advantages: Easy debugging, Faster testing, No separate compilation needed.

3. High-Level Language: Python hides hardware details from programmers and focuses on logic development.

4. Dynamically Typed Language: Variable types are identified automatically.

5. Object-Oriented Language: Python supports: Classes, Objects, Inheritance, Polymorphism, Encapsulation

6. Portable Language: Python programs can run on: Windows, Linux, macOS without changing the code.

7. Open Source Language: Python is freely available and its source code can be modified.

8. Large Standard Library: Python provides huge libraries for: Mathematics, Database connectivity, Networking, Artificial Intelligence, File handling.

Origin of Python

Python was developed by **Guido van Rossum in the late 1980s** at CWI (Centrum Wiskunde & Informatica), Netherlands.

The first version of Python was released in: February 1991

Why Python Was Developed: Simplify programming, Improve code readability, Reduce development time, Increase programmer productivity.

The name Python was inspired by: "Monty Python's Flying Circus" which was a British comedy television show.

Python Versions: Python has evolved continuously since its creation. Different versions of Python were released to improve performance, add new features, fix bugs, and enhance security. Each version introduced improvements that made Python more powerful and user-friendly.

Version	Features
Python 1.0	Basic features
Python 2.x	Improved libraries
Python 3.x	Modern and advanced version

Comparison of Python with Other Languages

Python	C
Simple syntax	Complex syntax
Interpreted	Compiled
Automatic memory management	Manual memory management
Slower execution	Faster execution

Python	Java
Dynamically typed	Statically typed
Less code	More code
Easy syntax	Complex syntax
Faster development	Slower development

Python	C++
Simple syntax	Complex syntax
Easy debugging	Difficult debugging
No pointers	Uses pointers
Interpreted	Compiled

Comments in Python

Comments are explanatory statements written inside programs to improve readability and understanding.

Comments are ignored during execution.

Importance of Comments

Comments help in: Documentation, Code understanding, Maintenance, Debugging.

Types of Comments

1. Single-Line Comment

Uses # symbol.

Example:

```
# This is a comment  
print("Python")
```

2. Multi-Line Comment

Uses triple quotes.

Example:

```
"""  
This is  
multi-line  
comment  
"""
```

Variables and Assignment

Variable: A variable is a named memory location used to store data values during execution.

Variables help programs store and manipulate data.

Characteristics of Variables

1. Variables store values temporarily.
2. Variable values can change during execution.
3. Python automatically identifies variable type.
4. Variables point to memory locations.

Creating Variables

Example:

```
x = 100  
name = "Veer"
```

Assignment Operator:

The = symbol assigns values to variables.

Example:

```
x = 10
```

Multiple Assignment

Example:

```
a = b = c = 5
```

Multiple Variable Assignment

Example:

```
x, y, z = 1, 2, 3
```

Dynamic Typing: Python automatically determines data type.

Example:

```
x = 10  
x = "Hello"
```

Identifiers in Python

Identifiers are names used to identify: Variables, Functions, Classes, Objects, Modules.

Rules for Identifiers

1. Must begin with letter or underscore.
2. Cannot start with numbers.
3. Cannot contain spaces.
4. Cannot use special symbols except underscore.
5. Keywords cannot be used.
6. Identifiers are case-sensitive.

Valid Identifiers

```
student_name  
marks1  
_total
```

Invalid Identifiers

```
1name  
user-name  
class
```

Keywords in Python: Keywords are reserved words with predefined meanings.

Keywords: False, None, True, and, as, assert, async, await, break, class, continue, def, del, elif, else, except, finally, for, from, global, if, import, in, is, lambda, nonlocal, not, or, pass, raise, return, try, while, with, yield, match, case.

Basic Style Guidelines (PEP 8)

PEP 8 stands for Python Enhancement Proposal 8. It defines coding standards for writing readable Python programs.

Importance of PEP 8

1. Improves readability.
2. Maintains consistency.
3. Simplifies maintenance.
4. Improves teamwork.

Important Style Guidelines

1. Proper Indentation: Python uses indentation to define code blocks.

```
if x > 0:  
    print("Positive")
```

Standard indentation:

- 4 spaces

2. Meaningful Variable Names

student_name ✓

sn ✗

3. Proper Spacing

x = y + z ✓

x=y+z ✗

Python Objects

Everything in Python is treated as an object.

Examples: Numbers, Strings, Lists, Functions.

Characteristics of Objects

Each object contains:

1. Identity
2. Type
3. Value

1. Identity: Identity refers to memory address.

```
x = 10
print(id(x))
```

2. Type: Type defines category of object.

```
print(type(x))
```

3. Value: Value represents actual stored data.

Mutable and Immutable Objects

Mutable	Immutable
List	Integer
Dictionary	String
Set	Tuple

Standard Types in Python

Python provides several built-in standard data types.

1. Numeric Types: Numeric types store numbers.

Types: Integer, Float, Complex

Example:

```
x = 10
y = 5.5
z = 2 + 3j
```

2. Boolean Type: Boolean type stores:

- True
- False

Example:

```
isActive = True
```

3. Sequence Types: Sequence types store ordered collections.

Types: String, List, Tuple.

Example:

```
name = "Python"
numbers = [1,2,3]
data = (4,5,6)
```

4. Set Type: Stores unique unordered elements.

```
s = {1,2,3}
```

5. Dictionary Type: Stores key-value pairs.

```
student = {"name":"Veer","age":21}
```

Other Built-in Types

1. None Type: Represents absence of value.

```
x = None
```

2. File Type: Used for handling files.

```
file = open("test.txt")
```

3. Function Type: Functions are treated as objects.

```
def add():  
    pass
```

Internal Types in Python

Internal types are used internally by Python interpreter.

Examples: Code objects, Frame objects, Traceback objects

Code Object

Stores compiled bytecode instructions.

Frame Object

Stores execution details during runtime.

Traceback Object

Stores exception and error information.

```
try:  
    x = 1/0  
except Exception as e:  
    print(e)
```

Operators in Python

Operators are symbols used to perform operations on variables and values.

Types of Operators

1. Arithmetic Operators

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus
//	Floor Division
**	Exponent

Example:

```
a = 10  
b = 3  
print(a+b)
```

2. Comparison Operators

Operator	Meaning
=	Equal
!=	Not equal
>	Greater
<	Less
>=	Greater equal
<=	Less equal

3. Logical Operators

Operator	Meaning
and	Logical AND
or	Logical OR
not	Logical NOT

4. Assignment Operators

Operator	Example
=	x = 5
+=	x += 2
-=	x -= 2

5. Membership Operators

Operator	Meaning
in	Present
not in	Not present

6. Identity Operators

Operator	Meaning
is	Same object
is not	Different object

Built-in Functions in Python

Built-in functions are predefined functions provided by Python interpreter.

Common Built-in Functions

Function	Purpose
print()	Displays output
input()	Takes input
len()	Returns length
type()	Returns type
int()	Converts to integer
str()	Converts to string
max()	Returns maximum
min()	Returns minimum

Examples

print()

```
print("Hello")
```

len()

```
name = "Python"  
print(len(name))
```

type()

```
x = 10  
print(type(x))
```

Introduction to Numbers

Numbers are one of the most important data types in Python. Python supports numerical operations such as:

- Addition
- Subtraction
- Multiplication
- Division
- Scientific calculations

Python supports:

1. Integers
2. Floating Point Numbers
3. Complex Numbers

Integers in Python

Integers are whole numbers without decimal points.

Examples:

10
-25
0

Example:

```
x = 100
```

Floating Point Real Numbers

Floating point numbers contain decimal points.

Examples:

10.5
-2.75
3.14

Example:

```
x = 10.5
```

Complex Numbers

Complex numbers contain:

- Real part
- Imaginary part

General form:

$a + bj$

Example:

```
x = 2 + 3j
```

Sequences in Python

Sequences are ordered collections of elements.

Characteristics:

1. Ordered collection
2. Supports indexing
3. Supports slicing
4. Iterable

Types: Strings, Lists, Tuples

Strings in Python

A string is a sequence of characters enclosed within quotation marks.

Characteristics of Strings

1. Immutable
2. Ordered
3. Indexed
4. Iterable

Types of Strings

Single Quote: Used to create a simple string using single quotes.

'Python'

Double Quote: Used to create a string using double quotes.

"Python"

Triple Quote: Used to create multi-line or long strings easily.

"""Python"""

String Indexing

Example:

```
name = "Python"  
print(name[0])  
Output:
```

P

String Slicing

Example:

```
print(name[1:4])  
Output:
```

yth

String-only Operators

1. Concatenation Operator (+)

Joins strings together.

Example:

"Hello" + "World"

2. Repetition Operator (*)

Repeats strings multiple times.

Example:

```
"Hi" * 3
```

3. Membership Operators

Example:

```
"P" in "Python"
```

String Built-in Methods

Method	Purpose
upper()	Converts uppercase
lower()	Converts lowercase
replace()	Replaces text
split()	Splits string
strip()	Removes spaces

Example of String Methods

```
name = "python"

print(name.upper())
print(name.lower())
```

Special Features of Strings

1. Immutable Nature

Strings cannot be modified after creation.

2. Unicode Support

Python supports Unicode characters.

Example:

```
text = "नमस्ते"
```

3. Multi-line Strings

Triple quotes support multiple lines so it can store and display characters from many languages.

Example:

```
text = """Python
is
easy"""
```

4. Escape Characters

Escape Character	Meaning
\n	New line
\t	Tab
\	Backslash

Example:

```
print("Hello\nWorld")
```

Memory Management in Python

Memory management refers to allocation and deallocation of memory during execution.

Python automatically manages memory using:

- Reference counting
- Garbage collection

Reference Counting

Every object maintains reference count.

When reference count becomes zero:

- Memory is automatically released.

Garbage Collection

Garbage collector removes unused objects automatically from memory.

Advantages:

1. Prevents memory leaks
2. Improves efficiency
3. Optimizes memory usage

Python Application Examples

1. Web Development

Python is used for developing websites and web applications using frameworks such as Django and Flask.

2. Artificial Intelligence

Python is widely used in artificial intelligence for creating intelligent systems such as chatbots and recommendation systems.

3. Machine Learning

Python supports machine learning through libraries such as Scikit-learn and TensorFlow.

4. Data Science

Python is used for data analysis, visualization, and statistical processing of large datasets.

5. Automation

Python automates repetitive tasks such as file handling, report generation, and email sending.

6. Scientific Applications

Python is used in scientific research and engineering applications for simulations and mathematical computations.

7. Cyber Security

Python is used in ethical hacking, penetration testing, and network security analysis.

8. Game Development

Python supports game development using libraries such as Pygame.

9. Desktop GUI Applications

Python is used for developing graphical desktop applications using Tkinter and PyQt.

10. Cloud Computing

Python is used in cloud automation, server management, and deployment scripting.

Unit-2 Lists and Dictionaries:

Built-in Functions, List type built in Methods, Special Features of Lists, Tuples, Tuple Operators and Built-in Functions, Special Features of Tuples, Introduction to Dictionaries, Built-in Functions, Built-in Methods, Dictionary Keys, Conditionals and Loops: if statement, else Statement, elif Statement, while Statement, for Statement, break Statement, continue Statement, pass Statement, else Statement.

Introduction to Data Structures in Python

In programming, a large amount of data needs to be stored, processed, and managed efficiently. Storing every value in separate variables becomes difficult when the amount of data increases. To solve this problem, programming languages provide data structures.

A data structure is a special way of organizing and storing data in memory so that it can be accessed and manipulated efficiently.

Python provides several built-in data structures such as:

- Lists
- Tuples
- Dictionaries
- Sets

Among these, lists, tuples, and dictionaries are the most commonly used collection data types.

These data structures are extremely important because they help in:

- Efficient data storage
- Data processing
- Fast searching
- Easy modification
- Better memory utilization
- Program optimization

Data structures are widely used in:

- Database systems
- Machine learning
- Artificial intelligence
- Scientific computing
- Web applications
- Data analysis

Introduction to Lists

A list is one of the most flexible and powerful data structures in Python. A list is an ordered collection of elements enclosed within square brackets [].

Lists are used to store multiple values in a single variable. They are mutable, meaning their elements can be modified after creation.

Lists can contain:

Integers

Strings

Floating point numbers

Boolean values

Objects

Other lists

Definition of List

A list is a mutable, ordered sequence of elements used for storing multiple data values together.

Characteristics of Lists

1. Ordered Collection: Lists preserve insertion order. Elements remain stored in the same sequence in which they were added.

2. Mutable Nature: List elements can be modified, added, or removed after creation.

3. Allows Duplicate Values: Lists can contain repeated values.

Example:

```
numbers = [1,1,2,2]
```

4. Heterogeneous Data Storage: Lists can store multiple data types together.

Example:

```
data = [10,"Python",5.5,True]
```

5. Dynamic Size: List size automatically changes during execution.

Creating Lists: Lists are created using square brackets.

Example:

```
fruits = ["Apple","Mango","Orange"]
```

```
print(fruits)
```

Output: ['Apple', 'Mango', 'Orange']

Empty List

Example:

```
data = []  
print(data)
```

Output: []

Nested Lists: Lists can contain other lists.

Example:

```
matrix = [[1,2],[3,4]]  
print(matrix)
```

Output: [[1, 2], [3, 4]]

Accessing List Elements: List elements are accessed using indexing.

Positive Indexing: Indexing starts from 0.

Example:

```
numbers = [10,20,30]  
print(numbers[0])  
print(numbers[1])
```

Output:10

20

Negative Indexing: Negative indexing starts from end.

Example:

```
numbers = [10,20,30]  
print(numbers[-1])
```

Output: 30

List Slicing: Slicing extracts specific portion of list.

Syntax:

```
list[start:end]
```

Example:

```
numbers = [1,2,3,4,5]  
print(numbers[1:4])
```

Output: [2, 3, 4]

Built-in Functions for Lists: Python provides several built-in functions for lists.

Function	Description
len()	Returns number of elements
max()	Returns largest element
min()	Returns smallest element
sum()	Returns total sum
sorted()	Returns sorted list
list()	Converts sequence into list

len() Function: The len() function returns total number of elements.

Example:

```
numbers = [1,2,3]  
print(len(numbers))
```

Output: 3

max() Function: The max() function returns largest element.

Example:

```
numbers = [10,50,20]  
print(max(numbers))
```

Output: 50

min() Function: The min() function returns smallest element.

Example:

```
numbers = [10,50,20]  
print(min(numbers))
```

Output: 10

sum() Function: The sum() function returns total sum.

Example:

```
numbers = [1,2,3]  
print(sum(numbers))
```

Output: 6

sorted() Function: The sorted() function sorts elements.

Example:

```
numbers = [5,1,3]
print(sorted(numbers))
```

Output: [1, 3, 5]

List Type Built-in Methods

Python provides many built-in methods specifically for lists.

1. append() Method: The append() method adds element at end of list.

Example:

```
numbers = [1,2]
numbers.append(3)
print(numbers)
```

Output: [1, 2, 3]

2. insert() Method: The insert() method inserts element at specified position.

Example:

```
numbers = [1,2]
numbers.insert(1,100)
print(numbers)
```

Output: [1, 100, 2]

3. extend() Method: The extend() method adds multiple elements.

Example:

```
numbers = [1,2]
numbers.extend([3,4])
print(numbers)
```

Output: [1, 2, 3, 4]

4. remove() Method: The remove() method removes specified element.

Example:

```
numbers = [1,2,3]
numbers.remove(2)
print(numbers)
Output: [1, 3]
```

5. pop() Method: The pop() method removes element using index.

Example:

```
numbers = [1,2,3]
numbers.pop()
print(numbers)
Output: [1, 2]
```

6. clear() Method: The clear() method removes all elements.

Example:

```
numbers = [1,2,3]
numbers.clear()
print(numbers)
Output: []
```

7. reverse() Method: The reverse() method reverses list order.

Example:

```
numbers = [1,2,3]
numbers.reverse()
print(numbers)
Output: [3, 2, 1]
```

8. sort() Method: The sort() method sorts original list.

Example:

```
numbers = [3,1,2]
numbers.sort()
print(numbers)
Output: [1, 2, 3]
```

9. count() Method: The count() method counts occurrences.

Example:

```
numbers = [1,1,2]  
print(numbers.count(1))
```

Output: 2

10. index() Method: The index() method returns position of element.

Example:

```
numbers = [10,20,30]  
print(numbers.index(20))
```

Output: 1

Special Features of Lists

1. Mutable Nature: Lists can be modified after creation.

Example:

```
numbers = [1,2,3]  
numbers[0] = 100  
print(numbers)
```

Output: [100, 2, 3]

2. Dynamic Memory Allocation: Lists automatically adjust memory size during execution.

3. Nested Lists: Lists can contain other lists.

Example:

```
matrix = [[1,2],[3,4]]  
print(matrix[0][1])
```

Output: 2

4. Supports Iteration: Lists can be traversed using loops.

Example:

```
numbers = [1,2,3]  
  
for x in numbers:  
    print(x)
```


Output:

```
1
2
3
```

Introduction to Tuples

A tuple is an ordered collection similar to lists, but tuples are immutable. Once created, tuple values cannot be changed.

Tuples are useful when data should remain fixed and protected.

Definition of Tuple

A tuple is an immutable ordered collection of elements enclosed within parentheses ().

Characteristics of Tuples

Ordered collection.

Immutable in nature.

Supports indexing and slicing.

Allows duplicate values.

Faster than lists.

Creating Tuples

```
numbers = (1,2,3)
```

```
print(numbers)
```

Output: (1, 2, 3)

Single Element Tuple

```
data = (5,)
```

```
print(data)
```

Output: (5,)

Tuple Indexing

Example:

```
numbers = (10,20,30)
```

```
print(numbers[1])
```

Output: 20

Tuple Slicing

Example:

```
numbers = (1,2,3,4)
```

```
print(numbers[1:3])
```

Output: (2, 3)

Tuple Operators

1. Concatenation Operator

Joins tuples.

Example:

```
a = (1,2)
```

```
b = (3,4)
```

```
print(a + b)
```

Output: (1, 2, 3, 4)

2. Repetition Operator

Repeats tuple.

Example:

```
print((1,2) * 3)
```

Output: (1, 2, 1, 2, 1, 2)

3. Membership Operator

Checks element existence.

Example:

```
print(1 in (1,2,3))
```

Output: True

Built-in Functions for Tuples

Function	Description
len()	Returns length
max()	Largest value
min()	Smallest value
sum()	Sum of values
tuple()	Converts into tuple

len() Function

Example:

```
numbers = (1,2,3)
print(len(numbers))
```

Output: 3

max() Function

Example:

```
numbers = (10,50,20)
print(max(numbers))
```

Output: 50

Tuple Built-in Methods

1. count() Method: Counts occurrences.

Example:

```
numbers = (1,1,2)
print(numbers.count(1))
```

Output: 2

2. index() Method: Returns position of element.

Example:

```
numbers = (10,20,30)
print(numbers.index(20))
```

Output: 1

Special Features of Tuples

1. Immutable Nature: Tuple values cannot be modified.

2. Faster Performance: Tuples execute faster than lists.

3. Data Security: Useful for fixed data storage.

4. Packing and Unpacking

Example:

```
data = (1,2,3)
```

```
a,b,c = data
```

```
print(a)  
print(b)  
print(c)
```

Output:

```
1  
2  
3
```

Introduction to Dictionaries

A dictionary is a collection of key-value pairs enclosed within curly braces { }.

Dictionaries are highly efficient for storing and retrieving data.

Definition of Dictionary

A dictionary is a mutable data structure that stores elements in key-value form.

Characteristics of Dictionaries

- Stores data as key-value pairs.

- Mutable in nature.

- Keys must be unique.

- Fast searching and retrieval.

- Supports heterogeneous data.

Creating Dictionaries

Example:

```
student = {  
    "name": "Veer",  
    "age": 21  
}
```

```
print(student)
```

Output:

```
{'name': 'Veer', 'age': 21}
```

Accessing Dictionary Values

Example:

```
student = {"name": "Veer"}
```

```
print(student["name"])
```

Output:

Veer

Adding Elements to Dictionary

Example:

```
student = {}
```

```
student["name"] = "Veer"
```

```
print(student)
```

Output:

```
{'name': 'Veer'}
```

Modifying Dictionary Values

Example:

```
student = {"age":21}
```

```
student["age"] = 22
```

```
print(student)
```

Output:

```
{'age': 22}
```

Removing Elements

Example:

```
student = {"name":"Veer","age":21}
```

```
del student["age"]
```

```
print(student)
```

Output:

```
{'name': 'Veer'}
```

Dictionary Built-in Methods

1. keys() Method

Returns all keys.

Example:

```
student = {"name":"Veer","age":21}
```

```
print(student.keys())
```

Output:

```
dict_keys(['name', 'age'])
```

2. values() Method

Returns all values.

Example:

```
print(student.values())
```

Output:

```
dict_values(['Veer', 21])
```

3. items() Method: Returns key-value pairs.

Example:

```
print(student.items())
```

Output:

```
dict_items([('name', 'Veer'), ('age', 21)])
```

4. get() Method: Returns value using key.

Example:

```
print(student.get("name"))
```

Output:

Veer

5. update() Method: Updates dictionary.

Example:

```
student.update({"age":22})
```

```
print(student)
```

Output:

```
{'name': 'Veer', 'age': 22}
```

6. pop() Method: Removes specified key.

Example:

```
student.pop("age")
```

```
print(student)
```

Output:

```
{'name': 'Veer'}
```

Conditionals and Loops

Control statements control execution flow of program.

They are mainly divided into:

Conditional statements

Looping statements

if Statement

The if statement executes block when condition is True.

Example:

```
x = 10  
  
if x > 5:  
    print("Greater")
```

Output:

Greater

else Statement

Executes when condition becomes False.

Example:

```
x = 2  
  
if x > 5:  
    print("Greater")  
else:  
    print("Smaller")
```

Output:

Smaller

elif Statement

Checks multiple conditions.

Example:

```
x = 0  
  
if x > 0:  
    print("Positive")  
elif x < 0:  
    print("Negative")  
else:  
    print("Zero")
```

Output:

Zero

while Statement

The while loop repeatedly executes code while condition remains True.

Example:

```
x = 1
```

```
while x <= 3:  
    print(x)  
    x += 1
```

Output:

```
1  
2  
3
```

for Statement

The for loop iterates through sequences.

Example:

```
for x in [1,2,3]:  
    print(x)
```

Output:

```
1  
2  
3
```

break Statement

Terminates loop immediately.

Example:

```
for x in range(5):  
    if x == 3:  
        break  
    print(x)
```

Output:

```
0  
1  
2
```

continue Statement

Skips current iteration.

Example:

```
for x in range(5):  
    if x == 2:  
        continue  
    print(x)
```

Output:

```
0  
1  
3  
4
```


pass Statement: Acts as placeholder and performs no operation.

Example:

```
for x in range(5):  
    pass  
  
print("Completed")
```

Output:

Completed

else Statement with Loops

The else block executes after successful completion of loop.

Example:

```
for x in range(3):  
    print(x)  
else:  
    print("Loop Finished")
```

Output:

```
0  
1  
2  
Loop Finished
```

Advantages of Lists

- Dynamic size allocation.
- Easy modification of elements.
- Supports multiple data types.
- Efficient iteration and manipulation.

Advantages of Tuples

- Faster execution than lists.
- Better memory efficiency.
- Provides data protection.

Advantages of Dictionaries

- Fast searching and retrieval.
- Efficient key-value storage.
- Easy updating and modification.
- Suitable for real-world applications.

Unit-3 Object, Classes and Files

Classes in Python, Principles of Object Orientation, Creating Classes, Instance Methods, Class variables, Inheritance, Polymorphism, Type Identification, File Objects, File Built-in Function, File Built-in Methods, File Built-in Attributes, Standard Files, Command-line Arguments, File System, File Execution, Persistent Storage Units.

Introduction to Object Oriented Programming (OOP)

Object Oriented Programming (OOP) is one of the most important programming paradigms used in modern software development. OOP organizes programs using objects and classes instead of only functions and procedures.

In traditional programming, programs become large and difficult to manage as complexity increases. Object-oriented programming solves this problem by dividing programs into small reusable units called objects.

Python fully supports Object-Oriented Programming and provides powerful features for creating object-oriented applications.

Object-oriented programming focuses mainly on:

- Objects
- Classes
- Data security
- Code reusability
- Modularity

OOP is widely used in:

- Web applications
- Banking systems
- Game development
- Artificial intelligence
- Software engineering
- GUI applications

Advantages of Object Oriented Programming:

- Improves code reusability.
- Simplifies program maintenance.
- Increases data security.
- Reduces code duplication.
- Makes large programs manageable.
- Provides modular structure.
- Supports real-world modeling.

Classes in Python

Classes are one of the most important concepts in Object-Oriented Programming (OOP). Python is an object-oriented programming language that supports classes and objects for developing modular, reusable, and efficient programs.

In procedural programming, large programs become difficult to manage because data and functions are handled separately. Classes solve this problem by combining data and methods into a single unit.

Definition of Class

A class is a blueprint or template used for creating objects.

A class defines:

- Attributes (variables)

- Methods (functions)

Objects created from a class contain actual data and behaviors.

Real-Life Example of Class

Example:

- Car is a class.

- BMW, Audi, and Toyota are objects.

Class defines common features:

- Color

- Speed

- Engine

Objects contain actual values of these features.

Characteristics of Classes

- Classes support object-oriented programming.

- Classes contain data and methods.

- Classes allow code reusability.

- Classes improve program organization.

- Classes support inheritance and polymorphism.

Syntax of Class

A class is created using: class keyword.

Syntax:

```
class ClassName:
```

statements

Example of Simple Class

```
class Student:  
    pass
```

Explanation of Program

Student is class name.

pass statement is used because class is empty.

Python allows empty classes.

Output

Class Created Successfully

Objects in Python

An object is an instance of a class.

Objects are created using class name.

Each object:

- Has separate memory location

- Contains data

- Accesses methods

Creating Objects

Example:

```
class Employee:
```

```
    name = "Veer"
```

```
e1 = Employee()
```

```
print(e1.name)
```

Output

Veer

Explanation

e1 is object of class Employee.

Object accesses class variable using dot operator.

Constructor in Python

A constructor is a special method automatically called when object is created.

Python constructor: `__init__()`

Example of Constructor

```
class Student:
```

```
    def __init__(self):  
        print("Constructor Called")
```

```
s = Student()
```

Output

Constructor Called

Parameterized Constructor

A constructor accepting arguments is called parameterized constructor.

Example:

```
class Student:
```

```
    def __init__(self,name,marks):  
        self.name = name  
        self.marks = marks
```

```
s = Student("Veer",90)
```

```
print(s.name)  
print(s.marks)
```

Output

Veer
90

self Keyword in Python

The self keyword refers to current object of class.

It is used to:

- Access instance variables

- Access instance methods

Example of self Keyword

```
class Student:
```

```
    def __init__(self,name):  
        self.name = name
```

```
s = Student("Veer")
```

```
print(s.name)
```

Output

Veer

Instance Variables

Instance variables belong to individual objects.

Each object has separate copy of instance variables.

Example of Instance Variables

```
class Student:
    def __init__(self,name):
        self.name = name

s1 = Student("Veer")
s2 = Student("Ram")

print(s1.name)
print(s2.name)
```

Output

Veer
Ram

Class Variables

Class variables are shared among all objects of class.

Only one copy exists.

Example of Class Variables

```
class Student:
    college = "ABC College"

s1 = Student()
s2 = Student()

print(s1.college)
print(s2.college)
```

Output

ABC College
ABC College

Difference Between Instance Variable and Class Variable

Instance Variable	Class Variable
Belongs to object	Belongs to class
Separate copy for each object	Shared by all objects
Defined using self	Defined inside class

Principles of Object Orientation

Object-oriented programming is based on four major principles:

Encapsulation

Abstraction

Inheritance

Polymorphism

1. Encapsulation: Encapsulation means combining data and methods into a single unit called class.

It protects data from unauthorized access.

Advantages of Encapsulation

Improves security.

Prevents accidental modification.

Increases maintainability.

Example of Encapsulation

```
class Student:
```

```
    def __init__(self):  
        self.name = "Veer"
```

```
s = Student()
```

```
print(s.name)
```

Output:

Veer

2. Abstraction: Abstraction means hiding internal implementation details and showing only essential features.

Example: ATM machine hides internal banking process.

Advantages of Abstraction

Reduces complexity.

Improves security.

Simplifies usage.

Example of Abstraction:

Python provides abstraction using abstract classes from: abc module.

```
from abc import ABC, abstractmethod
```

```
class ATM(ABC):
```

```
    @abstractmethod
```

```
    def withdraw(self):
```

```
        pass
```

```
class BankATM(ATM):
```

```
    def withdraw(self):
```

```
        print("Money Withdrawn Successfully")
```

```
a = BankATM()
```

```
a.withdraw()
```

Output:

Money Withdrawn Successfully

3. Inheritance: Inheritance is an important feature of Object-Oriented Programming that allows one class to acquire properties and methods of another class.

Inheritance supports:

Code reusability

Hierarchical classification

Easy maintenance

The existing class is called:

Parent class

Base class

Super class

The new class derived from existing class is called:

Child class

Derived class

Sub class

Real-Life Example of Inheritance

Vehicle → Parent class

Car → Child class

Car automatically inherits common properties of vehicle such as:

Speed
Engine
Fuel

Advantages of Inheritance

Code reusability.
Reduces code duplication.
Simplifies maintenance.
Improves extensibility.
Supports hierarchical relationships.

Syntax of Inheritance

```
class Parent:  
    pass  
  
class Child(Parent):  
    pass
```

Code Example of Inheritance

```
class Animal:  
  
    def sound(self):  
        print("Animals make sound")  
  
class Dog(Animal):  
  
    def bark(self):  
        print("Dog barks")  
  
d = Dog()  
  
d.sound()  
d.bark()
```

Output

Animals make sound
Dog barks

Types of Inheritance

Type	Description
Single Inheritance	One child inherits one parent
Multiple Inheritance	One child inherits multiple parents
Multilevel Inheritance	Inheritance chain
Hierarchical Inheritance	Multiple children inherit one parent
Hybrid Inheritance	Combination of inheritance types

Multiple Inheritance

```
class Father:
    def show_father(self):
        print("Father Class")

class Mother:
    def show_mother(self):
        print("Mother Class")

class Child(Father, Mother):
    def show_child(self):
        print("Child Class")

obj = Child()

obj.show_father()
obj.show_mother()
obj.show_child()
```

Multilevel Inheritance

```
class Grandparent:
    def grandparent_method(self):
        print("Grandparent Class")

class Parent(Grandparent):
    def parent_method(self):
        print("Parent Class")

class Child(Parent):
    def child_method(self):
        print("Child Class")

obj = Child()

obj.grandparent_method()
obj.parent_method()
obj.child_method()
```

Hierarchical Inheritance

```
class Parent:
    def parent_method(self):
        print("Parent Class")

class Child1(Parent):
    def child1_method(self):
        print("Child1 Class")

class Child2(Parent):
    def child2_method(self):
        print("Child2 Class")

obj1 = Child1()
obj2 = Child2()

obj1.parent_method()
obj1.child1_method()

obj2.parent_method()
obj2.child2_method()
```

Hybrid Inheritance

```
class A:
    def method_a(self):
        print("Class A")

class B(A):
    def method_b(self):
        print("Class B")

class C(A):
    def method_c(self):
        print("Class C")

class D(B, C):
    def method_d(self):
        print("Class D")

obj = D()

obj.method_a()
obj.method_b()
obj.method_c()
obj.method_d()
```

4. Polymorphism: Polymorphism means “many forms”. It allows same function or method to behave differently in different situations.

Polymorphism allows:

- Same function name
- Same method name
- Same operator

to behave differently in different situations.

Polymorphism improves flexibility and reusability in programs.

Real-Life Example of Polymorphism

Example:

Human behavior

A person behaves differently in different situations:

- Teacher in school
- Parent at home
- Friend in society

Same person → Different forms.

Advantages of Polymorphism

- Improves flexibility.
- Simplifies coding.
- Supports method overriding.
- Reduces code complexity.
- Improves code reusability.

Types of Polymorphism

Method Overriding

Operator Overloading

Method Overriding: When child class redefines method of parent class, it is called method overriding.

Child class redefines parent method.

Example:

class Parent:

```
def show(self):  
    print("Parent")
```

class Child(Parent):

```
def show(self):  
    print("Child")
```

c = Child()

c.show()

Output:

Child

Operator Overloading

Same operator performs multiple tasks.

Example:

```
print(10 + 20)  
print("Python" + "Language")
```

Output:

30

PythonLanguage

Instance Methods: Instance methods operate on object data.

They use:

self
parameter.

Example of Instance Methods

class Student:

```
def __init__(self,name):  
    self.name = name
```

```
def display(self):  
    print(self.name)
```

s = Student("Veer")

s.display()

Output: Veer

Type Identification

Python provides functions for identifying object type.

Functions:

`type()`

`isinstance()`

type() Function

Returns object type.

Example:

```
x = 10
```

```
print(type(x))
```

Output:

```
<class 'int'>
```

isinstance() Function

Checks object belongs to class or not.

Example:

```
x = 10
```

```
print(isinstance(x,int))
```

Output:

```
True
```

File Handling in Python

File handling is used for storing and retrieving data permanently. Variables store data temporarily in memory, but files store data permanently on storage devices.

Python provides built-in support for file handling.

Advantages of File Handling

Permanent data storage.

Data sharing between programs.

Large data management.

Backup and recovery support.

File Objects: A file object is created when file is opened using `open()` function.

Opening Files

Syntax:

```
open(filename,mode)
```

File Modes

Mode	Meaning
r	Read
w	Write
a	Append
x	Create
rb	Read binary
wb	Write binary

Example of Opening File

```
file = open("test.txt","w")
```

```
print(file)
```

Output: File object created

File Built-in Functions

open() Function

Opens file.

Example:

```
file = open("test.txt","w")
```

close() Function

Closes file.

Example:

```
file.close()
```

File Built-in Methods

Method	Purpose
read()	Reads file
readline()	Reads single line
write()	Writes data
writelines()	Writes multiple lines

seek()	Changes file position
tell()	Returns current position
close()	Closes file

write() Method

Example:

```
file = open("test.txt","w")  
file.write("Hello Python")  
file.close()
```

read() Method

Example:

```
file = open("test.txt","r")  
print(file.read())  
file.close()
```

Output:

Hello Python

readline() Method

Reads single line.

Example:

```
file = open("test.txt","r")  
print(file.readline())
```

File Built-in Attributes

Attribute	Purpose
name	File name
mode	File mode
closed	File status

Example of File Attributes

```
file = open("test.txt","r")  
  
print(file.name)  
print(file.mode)
```

Output:

```
test.txt  
r
```

Standard Files in Python

Python automatically provides three standard files:

Standard Input

Standard Output

Standard Error

Standard Input

Used for input operations.

Example:

```
name = input("Enter Name:")
```

Standard Output

Used for displaying output.

Example:

```
print("Python")
```

Standard Error

Used for displaying errors.

Command-line Arguments in Python

Command-line arguments are values passed to a Python program during execution through the command prompt or terminal.

Python uses the `sys.argv` list to access these arguments.

Syntax

```
import sys
```

```
print(sys.argv)  
    sys.argv[0] → Program name  
  
    sys.argv[1] → First argument  
  
    sys.argv[2] → Second argument
```

Example:

```
import sys  
  
print("Program Name:", sys.argv[0])
```



```
print("First Argument:", sys.argv[1])
```

Execution

```
python demo.py Hello
```

Output

Program Name: demo.py
First Argument: Hello

File System

A file system organizes files and directories on storage devices.

Python provides modules for file system operations.

Common modules:

- os
- shutil

Example of File System Operation

```
import os  
print(os.getcwd())
```

Output:

Current working directory displayed

File Execution in Python

Python files are executed using:

```
python filename.py
```

Example:

```
python test.py
```

Persistent Storage Units

Persistent storage stores data permanently even after program termination.

Examples:

Files, Databases, Cloud storage

Importance of Persistent Storage

Permanent data storage.

Data recovery support.

Large information management.

Data sharing across applications.

Unit-4 Regular Expression and Exception Handling

Regular Expression: Introduction/Motivation, Special Symbols and Characters for REs, REs and Python. What Are Exceptions? Exceptions in Python, Detecting and Handling Exceptions. Exceptions as Strings. Raising Exceptions, Assertions, Standard Exceptions.

Introduction to Regular Expressions

Regular Expressions (REs) are one of the most powerful tools used for pattern matching and text processing in programming. A regular expression is a sequence of special characters used to search, match, validate, and manipulate strings.

Regular expressions are extremely useful when dealing with:

- Text searching
- Data validation
- String manipulation
- Pattern matching
- Input verification

Python provides built-in support for regular expressions through the:

re
module.

Regular expressions are widely used in:

- Email validation
- Password checking
- Mobile number verification
- Search engines
- Data extraction
- Web scraping

Motivation for Regular Expressions

Before regular expressions, searching and validating text manually required lengthy and complex code. Regular expressions simplify these tasks by allowing programmers to define search patterns.

For example:

- Checking whether email is valid
- Finding phone numbers in text
- Extracting dates from files
- Searching keywords in documents

Regular expressions save:

Time

Coding effort

Complexity

Advantages of Regular Expressions

Simplifies text searching.

Reduces coding complexity.

Improves data validation.

Performs fast pattern matching.

Useful in large text processing applications.

REs and Python

Python provides built-in support for regular expressions using the:

re
module.

To use regular expressions, we import:

```
import re
```

Common Functions in re Module

Function	Purpose
match()	Matches pattern at beginning
search()	Searches pattern anywhere
findall()	Returns all matches
sub()	Replaces text
split()	Splits string

match() Function

The match() function checks pattern at beginning of string.

Example:

```
import re
```

```
result = re.match("Python","Python Language")
```

```
print(result)
```

Output:

```
<re.Match object>
```

search() Function

The search() function searches pattern anywhere in string.

Example:

```
import re

result = re.search("Language","Python Language")

print(result)
```

Output:

<re.Match object>

findall() Function

The findall() function returns all matching patterns.

Example:

```
import re

text = "Python is easy. Python is powerful."

result = re.findall("Python",text)

print(result)
```

Output:

['Python', 'Python']

sub() Function

The sub() function replaces matched text.

Example:

```
import re

text = "I like Java"

result = re.sub("Java","Python",text)

print(result)
```

Output:

I like Python

split() Function

The split() function splits string using pattern.

Example:

```
import re

text = "Python,Java,C++"
```

```
result = re.split(",",text)
```

```
print(result)
```

Output:

```
['Python', 'Java', 'C++']
```

Special Symbols and Characters for REs

Regular expressions use special symbols called meta-characters.

These symbols have special meanings.

1. Dot (.)

Matches any single character except newline.

Example:

```
import re
```

```
result = re.findall("P.t","Pat Pot Put",)
```

```
print(result)
```

Output:

```
['Pat', 'Pot', 'Put']
```

2. Caret (^)

Matches beginning of string.

Example:

```
import re
```

```
result = re.findall("^Python","Python Language")
```

```
print(result)
```

Output:

```
['Python']
```

3. Dollar (\$)

Matches end of string.

Example:

```
import re
```

```
result = re.findall("Language$","Python Language")
```

```
print(result)
```

Output:

```
['Language']
```

4. Star (*)

Matches zero or more occurrences.

Example:

```
import re

result = re.findall("Py*", "Pyyyy")

print(result)
```

Output:

```
['Pyyyy']
```

5. Plus (+)

Matches one or more occurrences.

Example:

```
import re

result = re.findall("Py+", "Pyyyy")

print(result)
```

Output:

```
['Pyyyy']
```

6. Question Mark (?)

Matches zero or one occurrence.

Example:

```
import re

result = re.findall("colou?r", "color colour")

print(result)
```

Output:

```
['color', 'colour']
```

7. Square Brackets []

Used for character sets.

Example:

```
import re

result = re.findall("[aeiou]", "Python Language")

print(result)
```

Output: ['o', 'a', 'u', 'a', 'e']

8. Range [a-z]

Matches characters within range.

Example:

```
import re

result = re.findall("[0-9]", "A1B2C3")

print(result)
```

Output:

```
['1', '2', '3']
```

9. Pipe Symbol (|)

Acts as OR operator.

Example:

```
import re

result = re.findall("Python|Java", "Python Java")

print(result)
```

Output:

```
['Python', 'Java']
```

10. Escape Character (\)

Used to remove special meaning.

Example:

```
import re

result = re.findall("\.", "file.txt")

print(result)
```

Output:

```
['.']
```

Common Special Sequences in REs

Sequence	Meaning
\d	Digit
\D	Non-digit
\w	Word character
\W	Non-word character
\s	Whitespace

\S	Non-whitespace
----	----------------

Example of \d

```
import re  
  
result = re.findall("\d","A1B2")  
  
print(result)
```

Output:

```
['1', '2']
```

Example of \w

```
import re  
  
result = re.findall("\w","Python123")  
  
print(result)
```

Output:

```
['P', 'y', 't', 'h', 'o', 'n', '1', '2', '3']
```

Real-life Applications of Regular Expressions

1. Email Validation: Regular expressions check whether email format is correct.
2. Password Validation: Checks password strength.
3. Mobile Number Verification: Ensures correct number format.
4. Search Engines: Used for text searching.
5. Data Extraction: Extracts useful information from files.

Introduction to Exception Handling

During program execution, errors may occur. These errors can terminate the program unexpectedly. To handle such situations, Python provides Exception Handling.

Exception handling helps programmers:

- Detect errors

- Prevent crashes

- Maintain normal execution

What Are Exceptions: An exception is an abnormal event or runtime error that interrupts normal flow of program execution.

Examples:

Division by zero

File not found

Invalid input

Index out of range

Examples of Exceptions

```
print(10/0)
```

Output:

ZeroDivisionError

Types of Errors in Python

1. Syntax Errors

Errors caused by incorrect syntax.

Example:

```
if True
    print("Python")
```

2. Runtime Errors

Errors occurring during execution.

Example:

```
print(10/0)
```

3. Logical Errors

Program runs but gives incorrect output.

Example:

```
print(2 + 2 * 2)
```

Exceptions in Python

Python provides built-in support for handling exceptions using:

try

except

finally

else

try Block

Contains risky code.

except Block

Handles exceptions.

Example of Exception Handling

```
try:  
    x = 10/0  
  
except ZeroDivisionError:  
    print("Division by zero not allowed")
```

Output:

Division by zero not allowed

Flow of Exception Handling

Code inside try executes.

If exception occurs, control moves to except block.

Exception is handled.

Program continues execution.

Handling Multiple Exceptions

Example:

```
try:  
    x = int("abc")  
  
except ZeroDivisionError:  
    print("Division Error")  
  
except ValueError:  
    print("Invalid Value")
```

Output:

Invalid Value

else Block in Exception Handling

The else block executes if no exception occurs.

Example:

```
try:  
    x = 10/2  
  
except:  
    print("Error")  
  
else:  
    print("No Error")
```

Output: No Error

finally Block

The finally block always executes whether exception occurs or not.

Example:

```
try:
    x = 10/0

except:
    print("Error")

finally:
    print("Program Ended")
```

Output:

```
Error
Program Ended
```

Detecting and Handling Exceptions

Python automatically detects runtime errors and converts them into exceptions.

Programmers handle exceptions using:

```
try
except
```

Benefits:

- Prevents abnormal termination.

- Improves reliability.

- Improves user experience.

Exceptions as Strings

Exceptions can be displayed as strings for user-friendly error messages.

Example:

```
try:
    x = 10/0

except Exception as e:
    print(str(e))
```

Output:

```
division by zero
```

Raising Exceptions

Python allows programmers to generate exceptions manually using:

```
raise
keyword.
```

Example of raise

```
age = -5

if age < 0:
    raise ValueError("Age cannot be negative")
```

Output:

ValueError: Age cannot be negative

Assertions in Python

Assertions are used for debugging and testing conditions.

Python uses:

```
assert
statement.
```

Example of Assertion

```
x = 10

assert x > 0

print("Valid")
```

Output:

Valid

Assertion Failure Example

```
x = -5

assert x > 0
```

Output:

AssertionError

Standard Exceptions in Python

Python provides many built-in standard exceptions.

Exception	Description
ZeroDivisionError	Division by zero
ValueError	Invalid value
TypeError	Invalid type
NameError	Variable not defined
IndexError	Invalid index
KeyError	Invalid dictionary key
FileNotFoundError	File missing
ImportError	Module import problem

ZeroDivisionError Example

```
print(10/0)
```

Output:

ZeroDivisionError

ValueError Example

```
int("abc")
```

Output:

ValueError

IndexError Example

```
numbers = [1,2]
```

```
print(numbers[5])
```

Output:

IndexError

FileNotFoundError Example

```
open("abc.txt")
```

Output:

FileNotFoundError

Advantages of Exception Handling

- Prevents abnormal program termination.

- Improves program reliability.

- Simplifies debugging.

- Improves user experience.

- Maintains normal execution flow.

Advantages of Regular Expressions

- Fast text searching.

- Simplifies validation.

- Reduces coding complexity.

- Powerful pattern matching support.

- Useful in real-world applications.

Unit-5 Database Interaction

SQL Database Connection using Python, Creating and Searching Tables, Reading and storing config information on database, Programming using database connections, Python Multithreading: Understanding threads, Forking threads, synchronizing the threads, Programming using multithreading.

Introduction to Database Interaction

In modern applications, huge amounts of data are generated and managed daily. Storing data only in variables or files is not sufficient for large-scale applications because data may become difficult to organize, retrieve, update, and secure.

To solve this problem, databases are used.

A database is an organized collection of related data stored electronically for easy access, management, and updating.

Python provides powerful support for database interaction, allowing programs to:

- Connect databases
- Store data
- Retrieve data
- Update records
- Delete records
- Manage transactions

Python can interact with various databases such as:

- MySQL
- SQLite
- Oracle
- PostgreSQL
- MongoDB

Database interaction is widely used in:

- Banking systems
- Online examination systems
- E-commerce websites
- Hospital management systems
- Student management systems

Advantages of Database Interaction

- Permanent data storage.
- Fast data retrieval.

- Easy data management.
- Better security and integrity.
- Multi-user data sharing.
- Efficient handling of large data.

SQL Database Connection Using Python

Python provides built-in modules and external libraries for connecting SQL databases.

Common Python database modules:

- sqlite3
- pymysql
- mysql.connector
- psycopg2

Among these, SQLite is most commonly used because it is built into Python.

What is SQLite?

SQLite is a lightweight relational database management system embedded inside Python.

Features:

- Serverless database.
- Easy to use.
- Portable database file.
- No separate installation required.

Importing sqlite3 Module

Example:

```
import sqlite3
```

Creating Database Connection: A database connection establishes communication between Python program and database.

Syntax:

```
sqlite3.connect("database_name")
```

Example of Database Connection

```
import sqlite3  
  
conn = sqlite3.connect("student.db")  
  
print("Database Connected")
```

Output:

Database Connected

Connection Object

The connection object represents database connection.

It is used for:

- Executing SQL queries

- Managing transactions

- Closing database

Creating Cursor Object

A cursor object is used to execute SQL commands.

Example:

```
cursor = conn.cursor()
```

Complete Example of Database Connection

```
import sqlite3  
  
conn = sqlite3.connect("college.db")  
  
cursor = conn.cursor()  
  
print("Connection Successful")
```

Output:

Connection Successful

Creating Tables in Database: A table stores data in rows and columns.

SQL command used:

CREATE TABLE

Example of Creating Table

```
import sqlite3  
  
conn = sqlite3.connect("student.db")  
  
cursor = conn.cursor()  
  
cursor.execute("""  
CREATE TABLE student(  
id INTEGER,  
name TEXT,  
marks INTEGER  
)  
""")  
  
print("Table Created")
```



```
conn.close()
```

Output:

Table Created

Inserting Data into Table

SQL command:

INSERT INTO

Example of Inserting Data

```
import sqlite3

conn = sqlite3.connect("student.db")

cursor = conn.cursor()

cursor.execute("""
INSERT INTO student
VALUES(1,'Veer',90)
""")

conn.commit()

print("Data Inserted")

conn.close()
```

Output:

Data Inserted

commit() Method: The commit() method permanently saves changes in database.

Without commit():

Changes may not be stored permanently.

Searching Tables in Database

Searching means retrieving required records from database.

SQL command:

SELECT

Example of Searching Table

```
import sqlite3

conn = sqlite3.connect("student.db")

cursor = conn.cursor()

cursor.execute("SELECT * FROM student")

rows = cursor.fetchall()
```

```
for row in rows:  
    print(row)
```

```
conn.close()
```

Output:

```
(1, 'Veer', 90)
```

fetchall() Method

Returns all rows from query result.

fetchone() Method

Returns single row.

Example:

```
row = cursor.fetchone()
```

WHERE Clause in SQL

Used for filtering records.

Example:

```
cursor.execute(  
"SELECT * FROM student WHERE marks > 80")
```

Updating Records

SQL command:

```
UPDATE
```

Example of Updating Data

```
import sqlite3
```

```
conn = sqlite3.connect("student.db")
```

```
cursor = conn.cursor()
```

```
cursor.execute("""  
UPDATE student  
SET marks = 95  
WHERE id = 1  
""")
```

```
conn.commit()
```

```
print("Record Updated")
```

```
conn.close()
```

Output:

Record Updated

Deleting Records

SQL command:

DELETE

Example of Deleting Data

```
import sqlite3

conn = sqlite3.connect("student.db")

cursor = conn.cursor()

cursor.execute("""
DELETE FROM student
WHERE id = 1
""")

conn.commit()

print("Record Deleted")

conn.close()
```

Output:

Record Deleted

Reading and Storing Configuration Information in Database

Configuration information refers to application settings stored in database.

Examples:

- Username
- Password
- Theme settings
- Application preferences
- Server information

Storing configuration data in database improves:

- Security
- Flexibility
- Centralized management

Example of Configuration Table

```
import sqlite3

conn = sqlite3.connect("config.db")

cursor = conn.cursor()
```

```
cursor.execute("""
CREATE TABLE settings(
key TEXT,
value TEXT
)
""")
```

```
conn.commit()
```

```
print("Configuration Table Created")
```

Output:

Configuration Table Created

Inserting Configuration Information

```
cursor.execute("""
INSERT INTO settings
VALUES('theme','dark')
""")
```

```
conn.commit()
```

Reading Configuration Information

```
cursor.execute(
"SELECT * FROM settings")
```

```
rows = cursor.fetchall()
```

```
for row in rows:
    print(row)
```

Output:

('theme', 'dark')

Advantages of Storing Configuration in Database

Centralized management.

Improved security.

Easy updating of settings.

Better scalability.

Programming Using Database Connections

Database programming means developing applications that interact with databases dynamically.

Python database programming involves:

Creating connection

Creating cursor

Executing SQL queries

Fetching results

Closing connection

Steps in Database Programming

Step	Description
1	Import database module
2	Establish connection
3	Create cursor
4	Execute query
5	Commit changes
6	Close connection

Example of Complete Database Program

```
import sqlite3

conn = sqlite3.connect("college.db")

cursor = conn.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS student(
id INTEGER,
name TEXT
)
""")

cursor.execute("""
INSERT INTO student
VALUES(1,'Veer')
""")

conn.commit()

cursor.execute(
"SELECT * FROM student")

rows = cursor.fetchall()

for row in rows:
    print(row)

conn.close()
```

Output:

```
(1, 'Veer')
```

Introduction to Python Multithreading

Multithreading is a technique where multiple threads execute simultaneously within a single program.

A thread is the smallest unit of execution inside a process.

Multithreading improves:

Performance

Responsiveness

CPU utilization

Real-Life Example of Multithreading

Example:

In a web browser:

One thread downloads data

Another thread displays graphics

Another thread plays audio

All tasks execute simultaneously.

Advantages of Multithreading

Faster execution.

Better resource utilization.

Improved application performance.

Supports parallel tasks.

Better user experience.

Understanding Threads

Process vs Thread

Process	Thread
Independent program	Part of process
Heavyweight	Lightweight
More memory usage	Less memory usage
Slower communication	Faster communication

Thread Lifecycle

A thread passes through:

New state

Runnable state

Running state

Waiting state

Terminated state

threading Module in Python

Python provides:

threading
module for multithreading.

Creating Threads

Example:

```
import threading

def display():
    print("Thread Running")

t = threading.Thread(target=display)

t.start()
```

Output:

Thread Running

Forking Threads

Forking means creating multiple threads for parallel execution.

Each thread performs separate task independently.

Example of Multiple Threads

```
import threading

def task1():
    print("Task 1 Running")

def task2():
    print("Task 2 Running")

t1 = threading.Thread(target=task1)
t2 = threading.Thread(target=task2)

t1.start()
t2.start()
```

Output:

Task 1 Running
Task 2 Running

Synchronizing Threads

Thread synchronization controls access to shared resources to prevent conflicts.

Without synchronization:

Data inconsistency may occur.

Race Condition

When multiple threads access shared resource simultaneously causing incorrect results.

Lock in Python

Python uses:

Lock()
for synchronization.

Example of Thread Synchronization

```
import threading

lock = threading.Lock()

def display():
    lock.acquire()
    print("Thread Locked")
    lock.release()

t = threading.Thread(target=display)
t.start()
```

Output:

Thread Locked

acquire() Method

Locks resource.

release() Method

Unlocks resource.

join() Method

Waits for thread completion.

Example:

```
t.join()
```

Programming Using Multithreading

Example of Multithreading Program

```
import threading
import time

def task(name):
```



```
for i in range(3):
    print(name)
    time.sleep(1)

t1 = threading.Thread(
    target=task,
    args=("Thread-1",)
)

t2 = threading.Thread(
    target=task,
    args=("Thread-2",)
)

t1.start()
t2.start()

t1.join()
t2.join()

print("Program Finished")
```

Output:

```
Thread-1
Thread-2
Thread-1
Thread-2
Thread-1
Thread-2
Program Finished
```

Daemon Threads

Daemon threads run in background and terminate automatically when main program ends.

Example:

```
t.daemon = True
```

Advantages of Database Programming

Permanent data storage, Fast data retrieval, Better security, Efficient handling of large data, Multi-user support.

Advantages of Multithreading

Improves application speed, Efficient CPU utilization, Better responsiveness, Supports parallel processing, Reduces execution time.

Applications of Multithreading

Web servers, Online games, Banking systems, Video streaming, Operating systems, Chat applications